

고급파이썬프로그래밍및자동화 - 연구 코드 개선 프로젝트

학과: 전자.정보공학부(전자공학)
과정: 석사과정
학번: 202550190
이름: 정유진

1. 개요

1.1 연구 코드 소개

본 연구는 CNN 기반으로 특징을 추출하여 이미지 특성을 파악하고, 이미지 픽셀마다의 3차원 좌표를 복원하여 카메라 로컬라이제이션을 수행하는 SCR (Scene Coordinate Regression) 을 수행하는 코드이다. ACE (Accelerated Coordinate Encoding)를 베이스라인으로 하며, 랜덤하게 이미지 마스크를 생성하는 ACE와 달리, 본 연구에서는 건물과 같은 정적 환경과 더불어 엣지 영역과 같은 고유 구조로부터 샘플링 마스크를 추출하도록 하여 좌표 복원 시 relocalization 속도와 성능을 개선하고자 한다.

해당 코드의 입력은 SCR을 수행하고자 하는 환경에 대한 다수의 이미지이며, 출력은 카메라 포즈값으로, evaluation 수행 시 카메라 GT (Ground Truth) 포즈에 대한 rotation 및 translation 오차로 측정된다.

본 연구에서 수행한 코드 개선은 패키지 주요 코드인 ace_trainer.py와 ace_network.py이며, 각각 전체 학습 파이프라인에 필요한 함수가 포함된 코드와, 모델의 구조를 정의하는 코드이다.

성능 개선을 목적으로 기존 베이스라인에 여러 모듈을 부착하고 수정하는 식으로 실험을 진행하면서 빠르게 실험을 진행하였다. 그 결과 수정 내용이 기존 코드에 더해져, 코드 작성 과정에서 한 함수가 수행하는 역할이 불분명해졌고, 코드 가독성이 떨어지는 문제점이 존재했다. 특히, ACE는 5분 이내에 모델을 학습하고 테스트까지 수행한다는 장점이 있는데, 새로운 모듈을 부착하는 과정에서 기존 베이스라인의 강점인 신속함과 효율적임 약해지는 것을 학습 시간 증가로 확인할 수 있었다.

따라서 본 과제를 수행하며 코드 리팩토링을 통해 코드 가독성을 수행하고, 베이스라인 대비 변경된 코드에서도 학습 시간을 단축하는 것이 주요 목적이다.

1.2. 실험 환경 및 실행 환경

본 연구는 다음 사양의 컴퓨터에서 실험을 진행하였다.

부품	세부 사항
CPU	INTEL CORE I7-10700 코멧레이크
GPU	NVIDIA GeForce RTX 3060 (VRAM=12GB)
RAM	48GB

표1. 실험 환경 요약

1.1에서 언급한 것과 같이 ACE의 경우 평가 항목이 Camera localization 결과로, 카메라의 포즈 오차 (회전 및 병진 오차) 측정 모듈을 제공한다. 또한 학습 버퍼 생성 시간 및 학습 시

간 / 평가 시간에 대해 시간을 측정하여 출력된다. 이에따라 코드 구조 개선에 따른 성능 평가는 원본 코드와 유사 성능일 때의 속도 개선을 평가하였다. 또한 ACE의 주요 특징 중 하나는 카메라 포즈를 측정하기 위해 RANSAC을 수행한다는 점인데, 이산적인 연산을 네트워크 형식으로 처리한 DSAC*를 활용하고 있다. 인코더와 dsac* 모두 사전 학습된 요소들을 활용하지만, RANSAC 특성 상 항상 일정한 값을 출력하기 어렵다는 부분에서 seed 고정에 어려움이 있다. 그 결과 동일 모델을 평가하여도 학습 과정에서 실험 정확도 결과에 미세 오차가 있을 수 있다. 이를 보완하기 위해 평가를 3번 진행하여 정확도와 학습 시간에 대한 평균 결과를 최종 결과로 제시하였다.

2. 문제점 분석

실험 코드는 정형화된 소프트웨어를 만드는 것이 목적이 아니라 가설을 빠르게 검증하는 것을 목적으로 하므로, 일반적으로 구조적인 완성도보다 동작 여부를 우선하게 된다. 이 과정에서 새로운 아이디어가 기존 코드 위에 더해졌다가, 제거되기를 반복된다. 그 결과 정량적으로 드러나지 않는 설계 문제가 누적된다. 이러한 문제의 대표적인 양상에는 연산, 메모리 비효율, 가독성 저하, 중복 코드, 책임 경계의 붕괴, 재현성 및 확장성 저하 등이 있으며 이외에 다수 존재한다. 본 코드에서도 다음과 같은 문제점이 존재했다.

- 성능적 측면에서의 문제점:

결과의 정확도를 우선하며 빠른 실험을 진행할 경우, 그 코드에 적용된 연산이 어떤 시간 /공간 복잡도로 수행되는지 충분히 고려하지 않는다. 이에따라 나타날 수 있는 문제는 크게 2가지가 있는데, 이는 각각 동일 입력에 대해 같은 결과를 매번 재 계산하는 문제와, 현재 필요한 것보다 더 많은 데이터를 미리 생성하여 메모리에 적제하는 문제가 있다. 이러한 문제는 단일 호출 수준에서는 비용의 비효율이 미미하지만, 학습과 같이 동일 연산이 여러 번 반복되는 경우에서는 비용이 누적되며 실행 시간 증가와 메모리 낭비로 이어질 수 있다. 또한 자료 구조 선택이나 평가 시점에 대한 고려가 없는 구현은, 입력 규모가 커질수록 비효율이 비선형적으로 확대될 위험성을 가진다. 본 연구 코드에서도 반복 연산의 중복과 더불어 학습 버퍼 구성단계에서의 과다 생성이 관찰되었으며, 이는 베이스라인의 신속성을 약화시키는 요인일 것이라고 추정한다.

- 구조적 측면에서의 문제점:

기능을 빠르게 덧붙이는 개발 방식은 핵심 알고리즘과 로깅, 시간 측정, 디버깅, 설정 관리를 한곳에 뒤섞기 쉽다. 또한 하나의 클래스나 함수가 설정값, 런타임 상태, 모델 생성, 실행 기록을 모두 떠안으면서 단일 책임 원칙이 무너질 수 있다. 이러한 구조에서는 부가 로직이 핵심 로직 사이에 삽입되어 가독성이 떨어지고, 같은 성격의 코드가 여러 위치에 중복되며, 부가 기능이 본래 연산의 의미를 흐리게 만들기도 한다. 나아가 설정과 상태가 한 객체에 결합되어 있으면, 실험 조건을 변경할 때 수정 범위가 넓어지고, 어떤 설정으로 실험이 수행되었는지 추적하기 어렵다는 문제점이 존재한다. 이는 동일 실험을 반복하거나 ablation 비교가 중요한 연구에서 치명적인 문제로 이어질 수 있다. 본 연구 코드에서도 측정과 기록 로직이 핵심 학습 로직과 결합되어 있으며, 학습을 담당하는 단일 클래스가 다수의 책임을 동시에 보유하고 있음을 확인할 수 있었다.

3. 적용한 수업 개념

3.1 해시 테이블과 메모이제이션

첫 번째 개선에서는 해시테이블 기반 매핑과 메모이제이션을 적용했다. 먼저 딕셔너리는 키를 해시 함수로 버킷에 사상하는 해시 테이블로, 단수 균등 해싱 가정하에 조회와 삽입에 대한 기대 비용은 평균 $O(1)$ 이다. 이 기대 비용은 적재율(식 1.)에 의존하며, 개방 주소법에서 평균 탐사 횟수는 식 2. 와 같이 증가한다. 파이썬에서 딕셔너리는 적재율이 일정 임계치를 넘으면 테이블을 재해싱하고 확장하여 적재율을 낮게 유지하므로, 개별 삽입이 가끔 $O(n)$ 의 재해싱을 유발하더라도 상환 분석 관점에서 $O(1)$ 을 보장한다.

$$\alpha = \frac{n}{m}$$

식 1. 적재율 정의

$$\alpha^* = \frac{1}{1 - \alpha}$$

식 2. 개방 주소법에서의 평균 탐사 횟수 증가율

메모이제이션은 deterministic 함수 f 의 계산 결과를 키로 저장하고, 동일 키가 다시 등장했을 경우 재계산을 조회로 대체하는 기법이다. 이 대체가 정당한 필요충분 조건은 f 가 동일 키에 대해 항상 동일한 결과를 반환해야 한다는 것이다. 본 문제에서 대상 연산은 일부 입력에만 의존하며, 그 입력이 학습 전 구간에서 불변했기 때문에 3.1의 방식을 적용하는 것이 타당했다.

3.2 데코레이터

파이썬에서 함수는 일급 객체로서 변수 바인딩, 인자 전달, 반환이 가능하며, 함수를 받아 함수를 반환하는 고차 함수의 한 형태로 데코레이터가 있다. `@Decorator` 구문은 정의 시점에서 평가되는 $f = \text{Decorator}(f)$ 으로, wrapping 호출 시점이 아니라 함수 바인딩 시점에 일어난다는 점이 특징이다.

3.3 평가 전략

Eager evaluation은 표현식을 바인딩 시점에 즉시 계산해 전체 결과를 구체화 하는 반면, Lazy evaluation은 값이 실제로 요구되는 시점까지 계산을 미루며, 이터레이터, 제너레이터가 그 대표적 구현으로 작업 집합을 유한하게 유지한다. 전체 데이터 중 일부를 선택적으로 값을 구성함으로써, 보조 공간을 줄이는 것에 집중한다. 그러나 지연 평가는 시간 복잡도 및 총 연산량을 줄이지 않으며, 총 작업량이 동일하다면 평가시점을 미루는 것만으로 시간 이득은 없고 오히려 호출 오버헤드가 추가될 수 있다.

3.4 데이터 클래스

데이터 클래스는 타입이 명시된 필드 선언으로부터 `__init__`, `__repr__`, `__eq__` 등을 자동 생성하는 도구이며, `frozen=True`를 지정하면 `__setattr__`, `__delattr__`가 예외를 발생시키도록 재정의되어 인스턴스가 불변이 되고, 불변 보장에 따라 `__hash__`가 함께 생성되어 값 의미론을 가진다. 설정 객체가 불변일 경우 학습 실행은 코드, 설정 객체, 시드의 deterministic한 함수가 되며, 이는 동일 입력에 대해 동일 결과를 보장하는 재현성의 토대가 된다. 반면 가변 네임

스페이스를 드래소 사용하면 실행 동중 설정이 변경될 여지가 있어 이 보장이 깨질 수 있다.

4. 개선 과정

4.1 문제 1 - 자료 구조 개선

베이스라인에 부착한 컨텍스트 정렬 모듈은 `model.forward`가 호출될 때마다 정규화 좌표 그리드를 `torch.linspace`로 매번 새로 생성했다. 그러나 학습 시 패치 해상도는 16×32 로 고정되어 있어, 해당 모델이 학습을 반복하는 횟수에 비례하여 완전히 동일한 그리드 텐서를 반복 할당한다는 문제점을 가지점을 가진다. 이는 그리드가 이미지의 크기에만 의존한다는 데이터 특성을 고려하지 않은 구현이므로, 본 개선에서는 (H, W, device)을 키로하는 딕셔너리 캐시를 도입한다. 또한 멤버십 테스트로 최초 1회만 계산, 저장한 뒤 이후에는 재사용하도록 하여 중복 연산을 제거하고자 하였다.

4.2 코드 구조 개선

본 코드의 경우, 반복적으로 활용되는 코드, 모듈 추가에 따른 실험 환경 고려, 그리고 수정 사항에 대한 정리가 되어있지 않다. 특히, 학습 파이프라인 전반에 `time.time()` 기반 시간 측정 코드가 버퍼 생성 / 에포크 / 전체 구간마다 흩어져있어 핵심 학습 로직과 섞여있는 문제가 있었다. 또한 CUDA 연산은 비동기 실행되므로, 동기화 없이 측정한 시간은 커널 실행만 반영하여 실제 GPU 연산 시간을 정확히 재치 못하는 문제점이 있었다. 이를 개선하기 위해, 측정 정책을 `@cuda_timeit` 데코레이터로 분리하였으며 `functools.wraps`로 원본 함수의 메타 데이터를 보존하고, 호출 전후 `torch.cuda.synchronize()`를 삽입하여 측정 정확성을 확보했으며, 누적시간을 데코레이터 상태로 보관한다. 이로써 중복 코드를 제거하고, 핵심 로직과 측정을 분리하여 가독성 및 함수의 역할을 분명히 분리하였다. 끝으로 불필요한 코드를 제거하였다.

4.3 데이터 적재 방식 개선

활용한 baseline의 핵심 contribution인 `create_training_buffer`의 경우, 이미지마자 `features`, `pose`, `intrinsics`를 전체 N행으로 생성하고, 확률 마스크를 이용해서 약 16%만 샘플링하여 버퍼에 적재한다. 즉 나머지 이미지 영역에 대해서는 생성 즉시 폐기되는 불필요한 객체이며, eager 적재에 따른 비효율이 발생하는 지점이다. 그러므로 해당 개선에서는 샘플 인덱스를 먼저 lazy 추출하고, 평탄화 인덱스를 (batch, row, col)로 역산하여 선택된 행만 advanced indexing하여 gather하도록 변경하였다. 전체 (N, C) 중간 텐서를 만들지 않으므로 반복마다 생성되는 중간 객체와 피크 메모리 감소를 기대한다.

4.4 구조 및 재현성 개선

학습기 클래스 `TrainerACE`의 생성자는 설정값, 런타임 상태, 모델 생성, 마스크 빌더, 로깅, 시각화를 한 곳에 포함하고 있는 클래스이다. 이는 단일 책임 원칙에 위배되어 한 클래스의 역할이 불분명하고, 실험 조건을 바꿀 때 수정범위가 넓으며 어떤 설정으로 학습했는지 추적하기 어렵다. 이를 개선하기 위해 설정을 `@dataclass(frozen=True)`기반 `RunConfig`로, 변경되는 런타임 정보를 `TrainerState`로 분리하여 설정/상태를 구분한다. 이를 통해 지현성과 유지보수성을 높이고, 실험 조건 변경시 수정 위치를 국소화 할 수 있도록 개선한다.

4. 최적화 결과

4.1 자료 구조 개선 결과

가설: 컨텍스트 정렬 모듈의 forward에서 매 호출 반복되는 정규화 좌표 그리드 생성을, 입력에 의존하는 디셔너리 캐시로 대체하면 중복 연산이 제거되어 학습 시간이 단축될 것이다.

결과: 학습 시간은 3회 평균 기준, 개선 전 $399.7 \pm 2.1s$, 개선 후 $401.1 \pm 2.1s$ 로 두 분포가 표준편차 범위 내에서 중첩되었다. 정확도는 동등 범위를 유지하였다.

	총 학습 시간(s)	10cm/5°	Median(°/cm)
개선 전	399.7 ± 2.1	13.4%	0.3/22.4
개선 후	401.1 ± 2.1	14.9%	0.3/23.0

표 2. 자료구조 개선 결과 표

검증: 측정 가능한 학습 시간 단축은 확인되지 않았다. (차이가 run 간 표준편차 이내). 정확도는 개선 전과 동등하여 동작은 보존되었다.

4.2 코드 구조 개선 결과 (데코레이터)

가설: 학습 파이프라인에 분산된 시각 측정 코드를 데코레이터로 통합하면 중복이 제거되고 측정 관심사가 핵심 로직에서 분리되며, 동기화 삽입으로 측정 정확성을 확보하되 추가 오버헤드는 무시 가능할 것이다.

결과: 분산되어 있던 `time.time()` 호출 6곳과 누적 로직을 `@cuda_timeit` 데코레이터 1개로 통합하였다. 정확도는 동등하였고, 동기화 추가에 따른 학습시간 변화는 노이즈 범위 내에 머물렀다.

	측정 코드	측정 정확성	총 학습 시간(s)
개선 전	<code>time.time()</code> 6곳 산재	비동기로 부정확	399.7
개선 후	데코레이터 1개로 통합	<code>synchronize</code> 적용	402.4

표 3. 코드 구조 개선 결과 표

검증: 측정 코드 중복 제거 및 관심사 분리를 달성하였고, 정확도를 유지하면서 오버헤드는 무시 가능한 수준이다.

4.3 데이터 적재 방식 개선 결과

가설: 학습 버퍼 구성 시 전체 N행을 생성한 뒤 일부만 사용하는 eager 방식을, 샘플 인덱스를 먼저 정하고 선택행만 모으는 lazy 방식으로 변경하면 반복당 중간 객체와 피크 메모리가 감소할 것이다.

결과: 반복당 중간 객체는 크게 감소하였으나, `buffer-fill` 구간의 피크 메모리는 거의 변하지 않았다. 정확도는 동등했다.

	반복당 중간 객체	materialize 행 수	buffer-fill peak
개선 전	143.9 MB	107,184 행	1516.2 MB
개선 후	8.2 MB	6,144 행	1503.5 MB

표 4. 데이터 적재 식방 개선 결과 표

검증: 중간 객체 생성은 가설대로 크게 감소하였으나, 피크 메모리 감소는 미미하여 가설을

부분적으로 충족한다.

4.4 구조 및 재현성 개선 결과

가설: 설정과 런타임 상태를 분리하면 재현성과 유지보수성이 향상되며, 연산 자체는 불변이므로 정확도는 유지될 것이다.

결과: 가변 네임스페이스에 산재하던 설정 접근을 불변 데이터클래스 단일 객체로 통합하고, 런타임 상태도 별도 객체로 분리하였다. 정확도는 동등 범위를 유지하였다.

	반복당 중간 객체	materialize 행 수
설정 현표	argparse Namespace	RunCofig
설정 접근	self.options. * 69곳 산재	self.config. * 단일 불변 객체
런타임 상태	설정과 혼재	TrainerState 4필드로 분리
학습 중 설정 변경	가능	불가
10cm/5° (정확도)	13.4 %	14.9 %

표 5. 구조 및 재현성 개선 결과 표

검증: 설정/상태 분리와 설정 불변화를 달성하였고, 정확도는 개선 전과 동등하게 유지되었다.

4.5 결과 요약

네 가지 개선을 모두 적용한 최종 코드와 개선 전 원본의 비교는 다음과 같다.

	개선 전	개선 후
총 시간 (s)	399.7± 2.1	403.0
버퍼 생성 / 학습 시간 (s)	self.options. * 69곳 산재	self.config. * 단일 불변 객체
10cm/5°, 5cm/5°	98.9 / 300.8	101.2 / 301/7
Median Error (°/cm)	13.4%, 1.5%	14.9 %, 2.6%
반복당 중간 객체	0.3 / 22.4	0.4 / 23.0
buffer-fill peak	1516.2 MB	1503.5 MB
설정 접근 구조	options 69곳	RunConfig 단일
측정 코드	time.time() 6rht	데코레이터 1개

표 6. 최종 결과 요약

요약: 정확도는 전 항목에서 개선 전과 동등하여 모든 변경이 동작을 보존하였다. 학습 시간은 개선 전후가 표준편차 범위 내에서 구분되지 않아 유의미한 변화가 없었다. 실측 가능한 자원 개선은 반복당 중간 객체 생성량이 감소에 집중되었으며, 피크 메모리는 거의 변하지 않았다. 구조적 측면에서는 측정, 설정 로직의 분리가 이루어졌다. 가설 검증 결과 2건 충족, 1건 부분 충족, 1건 기각으로 요약된다.

5. 결과 해석 및 한계

5.1 자료 구조 개선

학습 시간이 단축되지 않은 이유는, 캐싱으로 제거한 연산이 전체 학습의 병목이 아니었기 때문이다. 제거된 연산은 16×32 크기의 좌표 그리드를 재생성하는 것으로, 단일 호출 비용이 매우 작아 약 23,000회의 반복에 걸쳐 누적하더라도 run 간 표준 편차 아래에 머문다. 전체 실행 시간이 회귀 헤드의 forward, backward 등 다른 연산에 의해 지배되는 상황에서, 비지배적 연산을 제거하는 것은 e2e 시간에 유의미한 영향을 주지 못한다. 즉 이 개선은 동일 결과를 보장하는 국소적으로 올바른 최적화이지만, 그 이득이 전체 성능으로 이어지지 않는다는 것이다.

이는 본 개선이 한계인 동시에, 단일 측정에 근거해, 약 5초 단축으로 보였던 초기 관찰이 실제로는 노이즈였음을 드러낸 결과이다. 향후 구체적으로 실제 지배 연산을 먼저 식별한 뒤, 그 지점에 최적화를 적용하여 시간 단축을 진행해야 한다.

5.2 코드 구조 개선 (데코레이터)

데코레이터 적용이 가설을 충족한 이유는, 이 개선의 목적이 성능 향상이 아니라 측정에 초점을 맞췄기 때문이다. 분산된 측정 코드는 단일 데코레이터로 통합함으로써 중복이 제거되고 핵심 학습 로직과 측정 로직이 분리되었으며, 이는 정량 지표가 아니라 코드 구조의 질로 평가되는 개선이다. 한편 측정 정확성을 위해 삽입한 torch.cuda.synchronize()는 CPU가 GPU 연산 완료를 대기하게 하므로 미세한 실제 비용을 추가하지만, 전체 실행 시간 대비 비중이 작아 측정상 노이즈에 묻혔다.

다만 이러한 이러한 동기와 비용은 호출 빈도가 매우 높은 미세 함수에 적용할 경우 무시할 수 없을 정도로 누적될 수 있다는 점에서, 적용 범위에 대한 판단이 필요하다는 한계를 가진다. 향후에는 synchronize 대신 torch.cuda.Event 기반 시각 측정을 사용하면, 정확성을 유지하면서도 동기화로 인한 오버헤드를 줄일 수 있다.

5.3 데이터 적재 방식

중간 객체가 94.3% 감소했음에도 피크 메모리가 거의 변하지 않는 이유는 buffer-fill 구간의 메모리 최고치를 결정하는 연산이, 우리가 제거한 materialization이 아니기 때문이다. 한 반복에서 인코더, forward, PIDNet 분할 forward, 컨텍스트 맵 생성은 약 1.5GB의 활성화 메모리를 점유하는 반면, 지연 평가로 제거한 전체 행 단위의 중간 텐서는 수십 MB 규모로 그 최고치에 미치지 못한다. 이론적으로 해당 단계의 작업 집합 공간을 줄이지만, 전역 피크는 모든 단계에 대한 최댓값이므로, 최댓값이 다른 연산에서 발생하면 전역 피크는 변하지 않는다.

이를 통해 지연 평가가 항상 메모리 이득으로 직결되지 않는 것을 확인할 수 있으며, 피크가 다른 연산에 의해 지배될 때 지연 평가의 효과가 제한된다는 것을 파악할 수 있었다.

5.4 구조 및 재현성 개선

네 가지 개선을 관통하는 결론은 다음과 같다. 모든 변경은 정확도를 보존하였고, 코드 구조와 반복당 자원 사용을 개선하였으나, 학습 시간과 피크 메모리 같은 e2e 성능 지표는 변하지 않았다. 그 공통 원인은 각 최적화가 파이프라인의 지배적 비용 지점이 아닌 부분을 대상으로 했기 때문이며, 이는 국소적 개선이 전역적 개선으로 이어지지 않을 수도 있다는 것을 보여준다. 다시 말해, 본 연구의 실제 성능 병목은 베이스라인에 부착된 세그멘테이션, 컨텍스트 생성 모듈의 forward 연산에 있으며, 이는 향후 최적화 필요 지점으로 볼 수 있다.

그럼에도 본 과제의 개선은 두 가지 가치를 가진다. 첫째, 정확도 손실 없이 코드의 가독성, 재현성, 자원 효율을 개선했다는 점에서 구조적 품질과 성능 지표가 독립적인 의미를 가진다는 것이다. 둘째, 단일 측정에 근거한 성급한 주장이 반복 측정을 통해 노이즈로 판명된 과정은, 측정 없이 성능을 주장해서는 아되며 실제 병목이 무엇인지 먼저 진단해야 한다는 교훈을 제공한다. 결과적으로 본 연구는 연구 코드의 최적화에서 국소적 개선의 정당성과 그 한계를 함께 확인하였으며, 향후 병목 중심 최적화로 나아갈 토대를 마련하였다.